Discussion Point:

Although writing a program in assembly language is much easier than using machine code, many would argue that its use is no longer justified. Can you investigate the arguments for and against?

6.03 Symbolic, relative and absolute addressing

When considering how an assembler would convert an assembly language program into machine code it is necessary to understand the difference between symbolic, relative and absolute addressing. To explain these, we can consider a simple assembly language program which totals single numbers input at the keyboard. Table 6.01 shows the program as it would be written using symbolic addressing together with an explanation of each instruction.

Assembly language program using symbolic addressing	Explanation of each instruction
`IN`	A single number is input at the keyboard and its ASCII code is stored in the accumulator
`SUB #48`	This subtraction converts the ASCII code into the binary code for the number (see Task 6.01)
`STO MAX`	The number in the accumulator is stored at the address labelled MAX:
`LDM #0`	Loads zero into the accumulator
`STO TOTAL`	The zero in the accumulator is stored at the address labelled TOTAL:
`STO COUNT`	The zero in the accumulator is stored at the address labelled COUNT:
`STRTLP:IN`	A single number is input at the keyboard and its ASCII code is stored in the accumulator
`SUB #48`	This subtraction converts the ASCII code into the binary code for the number
`ADD TOTAL`	Adds the value at address labelled TOTAL: to the value in the accumulator and stores the sum in the accumulator
`STO TOTAL`	The number in the accumulator is stored at the address labelled TOTAL:
`LDD COUNT`	Loads the value stored at address COUNT: into the accumulator
`INC ACC`	Adds 1 to the value in the accumulator
`CMP MAX`	Compares the value in the accumulator with the value stored at address MAX:
`JPN STRTLP`	If the compared values are not equal the program jumps to the instruction labelled STRTLP:
`END`	The execution of the program has finished
`MAX:`	A labelled address where a value can be stored
`TOTAL:`	A labelled address where a value can be stored
`COUNT:`	A labelled address where a value can be stored

Table 6.01 An assembly program using symbolic addressing with explanations

The convention has been followed that a label is written with a following colon which is ignored when the label is referenced. Note how the code is dominated by the use of the accumulator.

TASK 6.01

Check the ASCII coding table to see why the subtraction in Table 6.01 works.

Addressing mode	Use of the operand
Immediate	The operand is the value to be used in the instruction; `SUB #48` is an example.
Direct	The operand is the address which holds the value to be used in the instruction; `ADD TOTAL` is an example.
Indirect	The operand is an address that holds the address which has the value to be used in the instruction.
Indexed	The operand is an address to which must be added the value currently in the index register (IX) to get the address which holds the value to be used in the instruction.

Table 6.06 Addressing modes

For immediate addressing there are three options for defining the value:

- #48 specifies the denary value 48
- #B00110000 specifies the binary equivalent
- #&30 specifies the hexadecimal equivalent

6.06 Assembly language instructions

We continue to consider a simple processor with a limited instruction set. The examples described here do not correspond directly to those found in the assembly language for any specific processor. Individual instructions will have a match in more than one real-life set. The important point is that these examples are representative. In particular, there are examples of the most common categories of instruction.

Data movement

These types of instruction can involve loading data into a register or storing data in memory. Table 6.07 contains a few examples of the format of the instructions with explanations.

Instruction opcode	Instruction operand	Explanation
`LDM`	`#n`	Immediate addressing. Load the number n to ACC.
`LDR`	`#n`	Immediate addressing. Load the number n to IX.
`LDD`	`<address>`	Direct addressing. Load the contents at the given address to ACC.
`LDI`	`<address>`	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC.
`LDX`	`<address>`	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
`MOV`	`<register>`	Move the contents of the accumulator to the given register (IX).
`STO`	`<address>`	Store the contents of ACC at the given address.

Table 6.07 Some instruction formats for data movement